1. Implementation of Fuzzy Logic Operations

```
import numpy as np

def fuzzy_union_or(A, B, operator='max'):

   if len(A) != len(B):
      raise ValueError("Fuzzy sets must have the same length (same
universe of discourse).")

   if operator == 'max':

      return np.maximum(A, B)

   else:
      raise NotImplementedError(f"Operator '{operator}' not supported for
Fuzzy Union.")

def fuzzy_intersection_and(A, B, operator='min'):

   if len(A) != len(B):
      raise ValueError("Fuzzy sets must have the same length (same
universe of discourse).")

   if operator == 'min':

      return np.minimum(A, B)

   else:
      raise NotImplementedError(f"Operator '{operator}' not supported for
Fuzzy Intersection.")

def fuzzy_complement_not(A):


   return 1 - A

U = np.array([1, 2, 3, 4, 5])
print(f"Universe of Discourse (U): {U}\n")
```

```
A = np.array([1.0, 0.8, 0.4, 0.1, 0.0])
B = np.array([0.0, 0.1, 0.3, 0.7, 1.0])

print("--- Original Sets ---")
print(f"Fuzzy Set A: {A}")
print(f"Fuzzy Set B: {B}\n")

# a. Fuzzy UNION (OR)
A_OR_B = fuzzy_union_or(A, B)
print("--- Fuzzy UNION (A OR B) ---")
print(f"Operation: max(mu_A(x), mu_B(x))")
# Example: max(A[0], B[0]) = max(1.0, 0.0) = 1.0
print(f"Result (A OR B): {A_OR_B}\n")

# b. Fuzzy INTERSECTION (AND)
A_AND_B = fuzzy_intersection_and(A, B)
print("--- Fuzzy INTERSECTION (A AND B) ---")
print(f"Operation: min(mu_A(x), mu_B(x))")
# Example: min(A[1], B[1]) = min(0.8, 0.1) = 0.1
print(f"Result (A AND B): {A_AND_B}\n")

# c. Fuzzy COMPLEMENT (NOT)
NOT_A = fuzzy_complement_not(A)
print("--- Fuzzy COMPLEMENT (NOT A) ---")
print(f"Operation: 1 - mu_A(x)")
# Example: 1 - A[2] = 1 - 0.4 = 0.6
print(f"Result (NOT A): {NOT_A}\n")

NOT_B = fuzzy_complement_not(B)
print("--- Fuzzy COMPLEMENT (NOT B) ---")
print(f"Operation: 1 - mu_B(x)")
print(f"Result (NOT B): {NOT_B}")
```

2. Design of Fuzzy Inference System (FIS)

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
```

```
import matplotlib.pyplot as plt

# --- 1. Define the Antecedent (Input) and Consequent (Output) Variables ---

# New Antecedent/Consequent objects hold universe variables and membership functions
# The universe (range of possible values) for each variable is defined.
# Universe for 'service' quality, ranging from 0 to 10
service = ctrl.Antecedent(np.arange(0, 11, 1), 'service')
# Universe for 'food' quality, ranging from 0 to 10
food = ctrl.Antecedent(np.arange(0, 11, 1), 'food')
# Universe for 'tip' percentage, ranging from 0 to 25%
tip = ctrl.Consequent(np.arange(0, 26, 1), 'tip')

# --- 2. Define Fuzzy Membership Functions (Fuzzification) ---

# We use simple triangular and trapezoidal membership functions (trimf, trapmf)

# Service quality fuzzy sets
service['poor'] = fuzz.trimf(service.universe, [0, 0, 5])
service['acceptable'] = fuzz.trimf(service.universe, [0, 5, 10])
service['excellent'] = fuzz.trimf(service.universe, [5, 10, 10])

# Food quality fuzzy sets
food['bad'] = fuzz.trapmf(food.universe, [0, 0, 1, 3])
food['decent'] = fuzz.trimf(food.universe, [1, 5, 9])
food['great'] = fuzz.trapmf(food.universe, [7, 9, 10, 10])

# Tip percentage fuzzy sets
tip['low'] = fuzz.trimf(tip.universe, [0, 0, 13])
tip['medium'] = fuzz.trimf(tip.universe, [0, 13, 25])
tip['high'] = fuzz.trimf(tip.universe, [13, 25, 25])

# Optional: Visualize the membership functions
# service.view()
# food.view()
# tip.view()
```

```
# --- 3. Define the Fuzzy Rules (Rule Base) ---

# Rules are defined in a natural language-like IF-THEN format
rule1 = ctrl.Rule(service['poor'] | food['bad'], tip['low'])
rule2 = ctrl.Rule(service['acceptable'], tip['medium'])
rule3 = ctrl.Rule(service['excellent'] & food['great'], tip['high'])
rule4 = ctrl.Rule(food['decent'] & service['poor'], tip['medium'])

# --- 4. Create the Control System and Simulation ---

# The ControlSystem object holds the fuzzy rules
tip_control = ctrl.ControlSystem([rule1, rule2, rule3, rule4])

# The ControlSystemSimulation object allows us to pass inputs and get
outputs
tipping_simulation = ctrl.ControlSystemSimulation(tip_control)

# --- 5. Fuzzify, Infer, Aggregate, and Defuzzify (The Process) ---

# Pass the input values to the simulation (Crisp Inputs)
# Example: Service is 6.5/10, Food is 9.8/10
tipping_simulation.input['service'] = 6.5
tipping_simulation.input['food'] = 9.8

# Compute the result (runs the entire FIS process: Fuzzification ->
Inference -> Defuzzification)
tipping_simulation.compute()

# Get the crisp output value
tip_amount = tipping_simulation.output['tip']

# --- 6. Display Results ---

print(f"Service Rating: 6.5/10")
print(f"Food Rating: 9.8/10")
print(f"*** Recommended Tip: {tip_amount:.2f}% ***")

# Optional: Visualize the final result
# The `tip.view` method shows the aggregated output fuzzy set
# and the resulting crisp value (vertical line)
```

```
tip.view(sim=tipping_simulation)
plt.show()

# --- Example 2: Testing different inputs ---
print("\n--- Example 2: Poor Service/Bad Food ---")
tipping_simulation.input['service'] = 2
tipping_simulation.input['food'] = 3
tipping_simulation.compute()
print(f"Recommended Tip: {tipping_simulation.output['tip']:.2f}%")
```

3. Defuzzification Techniques

```
import numpy as np
import skfuzzy as fuzz
import matplotlib.pyplot as plt

def demonstrate_defuzzification(universe, aggregated_mf):
    """
    Applies and compares five common defuzzification techniques to a
given
    aggregated membership function (MF).
    """

    # 1. Centroid (CoG/CoM) - Most common and robust
    # Calculates the center of gravity of the area under the curve.
    cog = fuzz.defuzz(universe, aggregated_mf, 'centroid')

    # 2. Bisector (BoA)
    # Finds the vertical line that divides the area under the curve into two
equal halves.
    boa = fuzz.defuzz(universe, aggregated_mf, 'bisector')

    # 3. Mean of Maximum (MoM)
    # Calculates the average of all points in the universe that have the
maximum membership value (height).
    mom = fuzz.defuzz(universe, aggregated_mf, 'mom')

    # 4. Smallest of Maximum (SoM)
    # Calculates the smallest value in the universe that has the maximum
```

```
membership value.
   som = fuzz.defuzz(universe, aggregated_mf, 'som')

   # 5. Largest of Maximum (LoM)
   # Calculates the largest value in the universe that has the maximum
membership value.
   lom = fuzz.defuzz(universe, aggregated_mf, 'lom')

   # --- Print Results ---
   print("--- Defuzzification Results ---")
   print(f"Centroid (CoG):       {cog:.4f}")
   print(f"Bisector (BoA):       {boa:.4f}")
   print(f"Mean of Maximum (MoM): {mom:.4f}")
   print(f"Smallest of Max (SoM): {som:.4f}")
   print(f"Largest of Max (LoM):  {lom:.4f}")

   # --- Plot Visualization ---
   plt.figure(figsize=(10, 6))
   plt.plot(universe, aggregated_mf, 'b', linewidth=2.5, label='Aggregated
Fuzzy Set')

   # Plot the results of each defuzzification method
   plt.axvline(cog, color='r', linestyle='--', label=f'Centroid ({cog:.2f})')
   plt.axvline(boa, color='g', linestyle='-.', label=f'Bisector ({boa:.2f})')
   plt.plot([mom, mom], [0, 1.0], 'k:', label=f'MoM ({mom:.2f})') # Using a
vertical line for MoM
   plt.plot([som, som], [0, 1.0], 'c:', label=f'SoM ({som:.2f})')
   plt.plot([lom, lom], [0, 1.0], 'm:', label=f'LoM ({lom:.2f})')

   plt.title('Comparison of Defuzzification Techniques')
   plt.ylabel('Membership Degree')
   plt.xlabel('Output Universe')
   plt.ylim(0, 1.1)
   plt.legend()
   plt.grid(True)
   plt.show()

# --- 1. Define the Universe of Discourse and Fuzzy Set ---

# The universe (range of possible output values, e.g., tip percentage 0 to
```

```
25)
X = np.arange(0, 26, 0.1)

# An example of an aggregated output fuzzy set (a trapezoid + a triangle)
# In a real FIS, this is the result of applying all rules (Inference) and
# combining the outputs (Aggregation).
# We simulate a skewed/complex aggregated set to show differences in
methods.
mf_1 = fuzz.trapmf(X, [0, 5, 8, 11])
mf_2 = fuzz.trimf(X, [9, 15, 25])

# The final aggregated set is usually the maximum (union) of all rule
consequences
# Note: Since the output is the result of aggregation, it is an array of
membership values.
aggregated_mf = np.fmax(mf_1 * 0.7, mf_2 * 1.0) # Assume mf_1 was
scaled down by rule strength 0.7

# Run the demonstration
demonstrate_defuzzification(X, aggregated_mf)
```

4. Implementation of Single-Layer Perceptron

```
import numpy as np
import matplotlib.pyplot as plt

# --- 1. Define the Activation Function (Step Function) ---
def step_function(weighted_sum):
  """
  The Perceptron uses a Heaviside step function as its activation.
  It returns 1 (activate) if the weighted sum is non-negative, else 0
(deactivate).
  """
  return 1 if weighted_sum >= 0 else 0

# --- 2. Define the Perceptron Class ---
class Perceptron:
  """
  Implements the core learning logic of a Single-Layer Perceptron.
```

```
    """
    def __init__(self, num_inputs, learning_rate=0.1, max_epochs=100):
        # Initialize weights (W) and bias (b). We start with small random
weights.
        # W has size (num_inputs) and b is a single scalar.
        self.weights = np.random.uniform(low=-0.5, high=0.5,
size=num_inputs)
        self.bias = np.random.uniform(low=-0.5, high=0.5, size=1)
        self.learning_rate = learning_rate
        self.max_epochs = max_epochs

        # History to track error over epochs
        self.errors = []

    def predict(self, inputs):
        """
        Calculates the weighted sum and applies the step function.
        Output = step( (W . X) + b )
        """
        # (W . X) is the dot product of weights and input features
        weighted_sum = np.dot(inputs, self.weights) + self.bias

        # Apply the activation function
        return step_function(weighted_sum)

    def train(self, training_inputs, labels):
        """
        Trains the perceptron using the Perceptron Learning Rule.
        Weights are updated only when a misclassification occurs.
        """
        print(f"--- Training Perceptron (Epochs: {self.max_epochs}, Rate:
{self.learning_rate}) ---")

        for epoch in range(self.max_epochs):
            total_error = 0

            # Iterate through each training example
            for inputs, label in zip(training_inputs, labels):

                # Forward Pass: Predict the output
```

```
            prediction = self.predict(inputs)

            # Calculate the error
            error = label - prediction
            total_error += abs(error)

            # Backward Pass: Update weights and bias only if error is non-
zero
            if error != 0:
                # Perceptron Update Rule:
                # W_new = W_old + LR * error * X
                # b_new = b_old + LR * error * 1
                self.weights += self.learning_rate * error * inputs
                self.bias += self.learning_rate * error * 1 # Bias update

        # Record error for tracking convergence
        self.errors.append(total_error)

        # Check for convergence: Stop if all samples are classified
correctly
        if total_error == 0:
            print(f" ✅ Converged successfully at Epoch {epoch + 1}.")
            break

        if (epoch + 1) % 10 == 0:
             print(f"Epoch {epoch + 1}/{self.max_epochs}, Total Error:
{total_error}")

    print("\n--- Training Complete ---")
    print(f"Final Weights: {self.weights}")
    print(f"Final Bias: {self.bias[0]:.4f}")

# --- 3. Prepare Data for AND Gate (A Linearly Separable Problem) ---

# Input data (A, B)
X_train = np.array([
  [0, 0], # Input 1
  [0, 1], # Input 2
  [1, 0], # Input 3
  [1, 1]  # Input 4
```

```
])

# Output labels (A AND B)
y_train = np.array([0, 0, 0, 1])

# --- 4. Run the Perceptron Model ---

# Initialize the Perceptron with 2 inputs (features)
perceptron = Perceptron(num_inputs=X_train.shape[1],
learning_rate=0.1, max_epochs=50)

# Train the model
perceptron.train(X_train, y_train)

# --- 5. Test the Trained Model ---

print("\n--- Testing Model Predictions ---")
test_cases = X_train
test_labels = y_train

for inputs, expected in zip(test_cases, test_labels):
  prediction = perceptron.predict(inputs)
  status = "Correct" if prediction == expected else "Incorrect"
  print(f"Input: {inputs}, Predicted: {prediction}, Expected: {expected}
({status})")

# Optional: Visualize the error history
plt.figure(figsize=(8, 4))
plt.plot(perceptron.errors, marker='o')
plt.title('Perceptron Training Error Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Total Misclassifications')
plt.grid(True)
plt.show()
```

5. Multilayer Perceptron using Back propagation Algorithm

```
import numpy as np

# --- 1. Define Activation and Derivative Functions (Sigmoid) ---

def sigmoid(x):
    """Sigmoid activation function: 1 / (1 + e^(-x))"""
    # Prevents overflow in e^(-x) for very large negative numbers
    x = np.clip(x, -500, 500)
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(output):
    """Derivative of the sigmoid function, calculated using the output of the sigmoid."""
    # Derivative is O * (1 - O), where O is the output of the sigmoid.
    return output * (1 - output)

# --- 2. Define the Multilayer Perceptron Class ---
class MLP_Backpropagation:
    """
    Implements a simple two-layer Multilayer Perceptron (Input -> Hidden -> Output).
    """
    def __init__(self, input_size, hidden_size, output_size, learning_rate=0.1, max_epochs=10000):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        self.learning_rate = learning_rate
        self.max_epochs = max_epochs

        # Initialize Weights and Biases randomly (W_ih: Input to Hidden, W_ho: Hidden to Output)
        # Weights should be initialized small to prevent saturation of the sigmoid function.
        self.W_ih = np.random.uniform(low=-0.5, high=0.5, size=(input_size, hidden_size))
        self.b_h = np.zeros((1, hidden_size)) # Bias for hidden layer

        self.W_ho = np.random.uniform(low=-0.5, high=0.5, size=(hidden_size, output_size))
```

```
        self.b_o = np.zeros((1, output_size)) # Bias for output layer

        self.errors = [] # To track mean squared error

    def forward_pass(self, X):
        """
        Calculates the output for a given input X.
        Z = WX + b; A = f(Z)
        """
        # Hidden Layer Calculation (Input -> Hidden)
        self.net_h = np.dot(X, self.W_ih) + self.b_h
        self.out_h = sigmoid(self.net_h)

        # Output Layer Calculation (Hidden -> Output)
        self.net_o = np.dot(self.out_h, self.W_ho) + self.b_o
        self.out_o = sigmoid(self.net_o)

        return self.out_o

    def backward_pass(self, X, y, out_o, out_h):
        """
        Calculates and applies weight updates based on the error.
        This is the core of the Backpropagation algorithm.
        """
        # --- A. Output Layer Error and Delta ---
        # Error = Target - Output
        error_o = y - out_o
        # Output Delta (d_o) = Error * f'(net_o)
        d_o = error_o * sigmoid_derivative(out_o)

        # --- B. Hidden Layer Error and Delta (Error Backpropagation) ---
        # Error_h = Delta_o * W_ho^T
        # The error is propagated back using the weights W_ho.
        error_h = d_o.dot(self.W_ho.T)
        # Hidden Delta (d_h) = Error_h * f'(net_h)
        d_h = error_h * sigmoid_derivative(out_h)

        # --- C. Weight and Bias Updates (Gradient Descent) ---

        # Update W_ho (Hidden to Output Weights)
```

```
        # dW_ho = out_h.T . d_o
        self.W_ho += self.out_h.T.dot(d_o) * self.learning_rate
        self.b_o += np.sum(d_o, axis=0, keepdims=True) * self.learning_rate

        # Update W_ih (Input to Hidden Weights)
        # dW_ih = X.T . d_h
        self.W_ih += X.T.dot(d_h) * self.learning_rate
        self.b_h += np.sum(d_h, axis=0, keepdims=True) * self.learning_rate

        return np.mean(error_o**2)

    def train(self, X_train, y_train):
        """
        Main training loop.
        """
        print(f"--- Training MLP (Hidden Size: {self.hidden_size}, Rate:
{self.learning_rate}) ---")

        for epoch in range(self.max_epochs):
            # 1. Forward Pass
            out_o = self.forward_pass(X_train)

            # 2. Backward Pass (Calculate Error and Update Weights)
            mse = self.backward_pass(X_train, y_train, out_o, self.out_h)

            self.errors.append(mse)

            if (epoch + 1) % 1000 == 0:
                print(f"Epoch {epoch + 1}/{self.max_epochs}, Mean Squared Error:
{mse:.6f}")

        print("\n--- Training Complete ---")

# --- 3. Prepare Data for XOR Gate (A Non-Linearly Separable Problem) ---

# Input data (A, B) - 4 samples, 2 features
X_train = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
```

```
    [1, 1]
])

# Output labels (A XOR B) - 4 samples, 1 output
y_train = np.array([
    [0],
    [1],
    [1],
    [0]
])

# --- 4. Run the MLP Model ---

# Model Configuration: 2 inputs, 4 hidden neurons, 1 output
mlp = MLP_Backpropagation(
    input_size=2,
    hidden_size=4,
    output_size=1,
    learning_rate=0.2,
    max_epochs=10000
)

# Train the model
mlp.train(X_train, y_train)

# --- 5. Test the Trained Model ---

print("\n--- Testing Model Predictions ---")

# Pass the training data through the trained network
predictions = mlp.forward_pass(X_train)

for inputs, prediction, expected in zip(X_train, predictions, y_train):
    # Apply a threshold of 0.5 to convert the sigmoid output (0 to 1) into a
binary prediction (0 or 1)
    predicted_class = 1 if prediction[0] >= 0.5 else 0
    status = "Correct" if predicted_class == expected[0] else "Incorrect"

    # Print prediction as a continuous value and its thresholded class
    print(f"Input: {inputs}, Output: {prediction[0]:.4f}, Predicted Class:
```

```
{predicted_class}, Expected: {expected[0]} ({status})")

# Optional: Visualize the error history
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 5))
plt.plot(mlp.errors)
plt.title('MLP Training Error (Mean Squared Error) Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('MSE')
plt.grid(True)
plt.show()
```

6. Implementation of Simple Neural Network (McCulloh Pitts model)

```
#Implementation of Simple Neural Network (McCulloh Pitts model)
import numpy as np

# --- 1. Activation Function (Threshold Logic) ---

def mcp_activation(net_input, threshold):
    """
    The MCP neuron uses a fixed threshold activation.
    Output = 1 (Fires) if net_input >= threshold, else 0 (Does not fire).
    """
    return 1 if net_input >= threshold else 0

# --- 2. The MCP Neuron Function ---

def mcp_neuron(inputs, weights, threshold):
    """
    Simulates the McCulloch-Pitts neuron calculation.
    net_input = Sum(input_i * weight_i)
    Output = mcp_activation(net_input, threshold)
    """
    # Convert inputs and weights to NumPy arrays for easy dot product
    inputs = np.array(inputs)
    weights = np.array(weights)
```

```
    # Calculate the net input (Weighted Sum)
    net_input = np.dot(inputs, weights)

    # Determine the output based on the fixed threshold
    output = mcp_activation(net_input, threshold)

    return net_input, output

# --- 3. Implementation of Logical Gates ---

def implement_or_gate():
    """Simulates the Logical OR gate (Output is 1 if EITHER input is 1)."""

    print("\n--- Implementing Logical OR Gate ---")

    # Fixed parameters for OR Gate:
    # We want (1*1) + (1*0) >= 1  -> 1 >= 1 (True)
    # We want (0*0) + (0*0) >= 1  -> 0 >= 1 (False)
    weights = [1, 1]  # Equal importance for both inputs
    threshold = 1     # Fire if at least one weighted input is 1

    test_cases = [
        ([0, 0], 0),
        ([0, 1], 1),
        ([1, 0], 1),
        ([1, 1], 1)
    ]

    print(f"Weights: {weights}, Threshold: {threshold}")
    print("Input (A, B) | Net Input | Output | Expected")
    print("-" * 38)

    for inputs, expected in test_cases:
        net_input, output = mcp_neuron(inputs, weights, threshold)
        print(f"  {inputs[0]},
{inputs[1]}   |  {net_input}     |   {output}  |  {expected}")

def implement_not_gate():
    """Simulates the Logical NOT gate (Output is the inverse of the input)."""
```

```
    print("\n--- Implementing Logical NOT Gate ---")

    # Fixed parameters for NOT Gate (Single Input):
    # We need a strong negative weight to inhibit firing when the input is 1.
    weights = [-1]    # Negative weight for the single input
    threshold = 0     # Needs to be low (non-positive)

    test_cases = [
        ([0], 1),
        ([1], 0)
    ]

    print(f"Weights: {weights}, Threshold: {threshold}")
    print("Input (A) | Net Input | Output | Expected")
    print("-" * 38)

    for inputs, expected in test_cases:
        net_input, output = mcp_neuron(inputs, weights, threshold)
        print(f"   {inputs[0]}  |   {net_input}     |   {output}  |  {expected}")

# --- 4. Run the Implementations ---

implement_or_gate()
implement_not_gate()
```

7. (A) Implementation of Genetic Algorithm (GA)

```
import numpy as np
import matplotlib.pyplot as plt

def fitness_function(x):
    return x * np.sin(10 * np.pi * x) + 1
POP_SIZE = 40
GENERATIONS = 100
X_MIN, X_MAX = -1.0, 2.0
CROSSOVER_PROB = 0.9
MUTATION_PROB = 0.2
```

```
MUTATION_STD = 0.1

np.random.seed(42)
population = np.random.uniform(X_MIN, X_MAX, POP_SIZE)
def tournament_selection(pop, fitness, k=3):
  selected = []
  for _ in range(len(pop)):
    idx = np.random.choice(len(pop), k, replace=False)
    selected.append(pop[idx[np.argmax(fitness[idx])]])
  return np.array(selected)

def arithmetic_crossover(p1, p2):
  alpha = np.random.rand()
  return alpha * p1 + (1 - alpha) * p2, alpha * p2 + (1 - alpha) * p1

def mutate(x):
  if np.random.rand() < MUTATION_PROB:
    x += np.random.normal(0, MUTATION_STD)
  return np.clip(x, X_MIN, X_MAX)
best_history = []
mean_history = []
for _ in range(GENERATIONS):
  fitness = fitness_function(population)
  best_history.append(np.max(fitness))
  mean_history.append(np.mean(fitness))
  parents = tournament_selection(population, fitness)
  offspring = []
  np.random.shuffle(parents)
  for i in range(0, POP_SIZE, 2):
    if np.random.rand() < CROSSOVER_PROB:
      c1, c2 = arithmetic_crossover(parents[i], parents[i + 1])
    else:
      c1, c2 = parents[i], parents[i + 1]
    offspring.extend([mutate(c1), mutate(c2)])
  population = np.array(offspring)
x = np.linspace(X_MIN, X_MAX, 500)

plt.figure()
plt.plot(best_history, label="Best Fitness")
plt.plot(mean_history, label="Mean Fitness")
```

```
plt.legend()
plt.show()

plt.figure()
plt.plot(x, fitness_function(x))
plt.scatter(population, fitness_function(population))
plt.show()
```

7 (B)

```
import random

# Objective function
def fitness(x):
  return x * x

# Generate initial population
def initialize_population(size):
  return [random.uniform(-10, 10) for _ in range(size)]

# Selection (Tournament Selection)
def selection(population):
  a, b = random.sample(population, 2)
  return a if fitness(a) < fitness(b) else b

# Crossover
def crossover(parent1, parent2):
  alpha = random.random()
  return alpha * parent1 + (1 - alpha) * parent2

# Mutation
def mutation(child, rate=0.1):
  if random.random() < rate:
    child += random.uniform(-1, 1)
  return child

# Genetic Algorithm
def genetic_algorithm():
  population_size = 20
```

```
    generations = 50

    population = initialize_population(population_size)

    for _ in range(generations):
        new_population = []
        for _ in range(population_size):
            p1 = selection(population)
            p2 = selection(population)
            child = crossover(p1, p2)
            child = mutation(child)
            new_population.append(child)
        population = new_population

    best = min(population, key=fitness)
    print("Best solution:", best)
    print("Minimum value:", fitness(best))

genetic_algorithm()
```

8. (A) Solving Travelling Salesman Problem using GA

```
import random
import numpy as np

# Distance matrix
distance = np.array([
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
])

NUM_CITIES = 4
POP_SIZE = 10
GENERATIONS = 100
MUTATION_RATE = 0.2

# Calculate total distance
```

```
def total_distance(tour):
  return sum(distance[tour[i], tour[(i + 1) % NUM_CITIES]]
          for i in range(NUM_CITIES))

# Fitness function
def fitness(tour):
  return 1 / total_distance(tour)

# Initial population
def create_population():
  return [random.sample(range(NUM_CITIES), NUM_CITIES)
        for _ in range(POP_SIZE)]

# Tournament selection
def selection(population):
  a, b = random.sample(population, 2)
  return a if fitness(a) > fitness(b) else b

# Ordered crossover (OX)
def crossover(p1, p2):
  start, end = sorted(random.sample(range(NUM_CITIES), 2))
  child = [-1] * NUM_CITIES
  child[start:end] = p1[start:end]
  pointer = end

  for city in p2:
    if city not in child:
      if pointer >= NUM_CITIES:
        pointer = 0
      child[pointer] = city
      pointer += 1
  return child

# Swap mutation
def mutation(tour):
  if random.random() < MUTATION_RATE:
    i, j = random.sample(range(NUM_CITIES), 2)
    tour[i], tour[j] = tour[j], tour[i]
  return tour
```

```
# Genetic Algorithm
population = create_population()

for _ in range(GENERATIONS):
    new_population = []
    for _ in range(POP_SIZE):
        parent1 = selection(population)
        parent2 = selection(population)
        child = crossover(parent1, parent2)
        child = mutation(child)
        new_population.append(child)
    population = new_population

# Best solution
best_tour = min(population, key=total_distance)
print("Best tour:", best_tour)
print("Minimum distance:", total_distance(best_tour))
```

8 (B)

```
import numpy as np
import random
import matplotlib.pyplot as plt

# Distance matrix (5 cities)
dist = np.array([
    [0, 2, 9, 10, 7],
    [2, 0, 6, 4, 3],
    [9, 6, 0, 8, 5],
    [10, 4, 8, 0, 6],
    [7, 3, 5, 6, 0]
])

NUM_CITIES = len(dist)
POP_SIZE = 50
GENERATIONS = 200
MUTATION_RATE = 0.2

# Fitness function
```

```
def total_distance(route):
  return sum(dist[route[i], route[i+1]] for i in range(NUM_CITIES-1)) +
dist[route[-1], route[0]]

def fitness(route):
  return 1 / total_distance(route)

# Initial population
def init_population():
  return [random.sample(range(NUM_CITIES), NUM_CITIES) for _ in
range(POP_SIZE)]

# Tournament selection
def selection(pop):
  k = 3
  selected = random.sample(pop, k)
  return max(selected, key=fitness)

# Order Crossover (OX)
def crossover(p1, p2):
  a, b = sorted(random.sample(range(NUM_CITIES), 2))
  child = [-1]*NUM_CITIES
  child[a:b] = p1[a:b]

  ptr = b
  for city in p2:
    if city not in child:
      if ptr == NUM_CITIES:
        ptr = 0
      child[ptr] = city
      ptr += 1
  return child

# Swap Mutation
def mutate(route):
  if random.random() < MUTATION_RATE:
    i, j = random.sample(range(NUM_CITIES), 2)
    route[i], route[j] = route[j], route[i]
  return route
```

```
    # GA Main Loop
    population = init_population()
    best_dist = []

    for _ in range(GENERATIONS):
        new_pop = []
        for _ in range(POP_SIZE):
            p1 = selection(population)
            p2 = selection(population)
            child = crossover(p1, p2)
            child = mutate(child)
            new_pop.append(child)
        population = new_pop
        best_route = min(population, key=total_distance)
        best_dist.append(total_distance(best_route))

    print("Best Route:", best_route)
    print("Minimum Distance:", total_distance(best_route))

    plt.plot(best_dist)
    plt.xlabel("Generations")
    plt.ylabel("Distance")
    plt.title("GA for TSP")
    plt.show()
```

9. Design of Adaptive Neuro-Fuzzy Inference System (ANFIS)

```
import numpy as np
import random
import matplotlib.pyplot as plt

# Distance matrix (5 cities)
dist = np.array([
    [0, 2, 9, 10, 7],
    [2, 0, 6, 4, 3],
    [9, 6, 0, 8, 5],
    [10, 4, 8, 0, 6],
    [7, 3, 5, 6, 0]
```

```
])

NUM_CITIES = len(dist)
POP_SIZE = 50
GENERATIONS = 200
MUTATION_RATE = 0.2

# Fitness function
def total_distance(route):
  return sum(dist[route[i], route[i+1]] for i in range(NUM_CITIES-1)) +
dist[route[-1], route[0]]

def fitness(route):
  return 1 / total_distance(route)

# Initial population
def init_population():
  return [random.sample(range(NUM_CITIES), NUM_CITIES) for _ in
range(POP_SIZE)]

# Tournament selection
def selection(pop):
  k = 3
  selected = random.sample(pop, k)
  return max(selected, key=fitness)

# Order Crossover (OX)
def crossover(p1, p2):
  a, b = sorted(random.sample(range(NUM_CITIES), 2))
  child = [-1]*NUM_CITIES
  child[a:b] = p1[a:b]

  ptr = b
  for city in p2:
    if city not in child:
      if ptr == NUM_CITIES:
        ptr = 0
      child[ptr] = city
      ptr += 1
  return child
```

```
# Swap Mutation
def mutate(route):
    if random.random() < MUTATION_RATE:
        i, j = random.sample(range(NUM_CITIES), 2)
        route[i], route[j] = route[j], route[i]
    return route

# GA Main Loop
population = init_population()
best_dist = []

for _ in range(GENERATIONS):
    new_pop = []
    for _ in range(POP_SIZE):
        p1 = selection(population)
        p2 = selection(population)
        child = crossover(p1, p2)
        child = mutate(child)
        new_pop.append(child)
    population = new_pop
    best_route = min(population, key=total_distance)
    best_dist.append(total_distance(best_route))

print("Best Route:", best_route)
print("Minimum Distance:", total_distance(best_route))

plt.plot(best_dist)
plt.xlabel("Generations")
plt.ylabel("Distance")
plt.title("GA for TSP")
plt.show()
```

10. Comparison of Hard and Soft Computing Techniques

```
# ---------------- HARD COMPUTING ----------------
# Exact rule-based decision (Deterministic)

def hard_temperature(temp):
    if temp < 25:
        return "Cold"
    else:
        return "Hot"


# ---------------- SOFT COMPUTING ----------------
# Approximate decision (Fuzzy-like logic)

def soft_temperature(temp):
    if temp <= 20:
        return "Cold"
    elif 20 < temp <= 30:
        return "Warm"
    else:
        return "Hot"


# ---------------- TEST VALUES ----------------
temperature = [18, 25, 28, 35]

print("Temperature | Hard Computing | Soft Computing")
print("-----------------------------------------")

for t in temperature:
    print(f"{t}°C        | {hard_temperature(t):13} | {soft_temperature(t)}")
```

11. Implement PSO and use it to find the global minimum of the Rastrigin function

```
import numpy as np

# Rastrigin function
def rastrigin(X):
   A = 10
   return A * len(X) + np.sum(X**2 - A * np.cos(2 * np.pi * X))

# PSO parameters
num_particles = 30
dimensions = 2
iterations = 100
w = 0.7      # inertia
c1 = 1.5     # cognitive
c2 = 1.5     # social

# Initialize particles
X = np.random.uniform(-5.12, 5.12, (num_particles, dimensions))
V = np.random.uniform(-1, 1, (num_particles, dimensions))

pbest = X.copy()
pbest_val = np.array([rastrigin(x) for x in X])

gbest = pbest[np.argmin(pbest_val)]
gbest_val = min(pbest_val)

# PSO loop
for _ in range(iterations):
   for i in range(num_particles):
      r1, r2 = np.random.rand(), np.random.rand()
      V[i] = (w * V[i] +
            c1 * r1 * (pbest[i] - X[i]) +
            c2 * r2 * (gbest - X[i]))
      X[i] = X[i] + V[i]

      fitness = rastrigin(X[i])
      if fitness < pbest_val[i]:
         pbest[i] = X[i]
```

```
            pbest_val[i] = fitness

    gbest = pbest[np.argmin(pbest_val)]
    gbest_val = min(pbest_val)

print("Global Minimum Position:", gbest)
print("Minimum Value:", gbest_val)
```

12. Use PSO to solve a simple load balancing problem: assign N independent tasks to M machines to minimize makespan.

```
# PSO for Load Balancing (Makespan Minimization) - Python

import numpy as np
import random

# ------------------------------
# Problem Parameters
# ------------------------------
N = 6                # number of tasks
M = 3                # number of machines
task_time = np.array([2, 4, 6, 3, 5, 7])

num_particles = 20
iterations = 50
w, c1, c2 = 0.7, 1.5, 1.5

# ------------------------------
# Fitness Function (Makespan)
# ------------------------------
def fitness(position):
    loads = np.zeros(M)
    for i in range(N):
        machine = int(round(position[i])) % M
        loads[machine] += task_time[i]
    return max(loads)

# ------------------------------
# PSO Initialization
```

```
# ------------------------------
particles = np.random.randint(0, M, (num_particles, N))
velocities = np.zeros((num_particles, N))

pbest = particles.copy()
pbest_fitness = np.array([fitness(p) for p in particles])

gbest = pbest[np.argmin(pbest_fitness)]
gbest_fitness = min(pbest_fitness)

# ------------------------------
# PSO Main Loop
# ------------------------------
for _ in range(iterations):
  for i in range(num_particles):
    r1, r2 = random.random(), random.random()
    velocities[i] = (w * velocities[i] +
                c1 * r1 * (pbest[i] - particles[i]) +
                c2 * r2 * (gbest - particles[i]))
    particles[i] = particles[i] + velocities[i]

    fit = fitness(particles[i])
    if fit < pbest_fitness[i]:
      pbest[i] = particles[i]
      pbest_fitness[i] = fit

  gbest = pbest[np.argmin(pbest_fitness)]
  gbest_fitness = min(pbest_fitness)

# ------------------------------
# Result
# ------------------------------
print("Best Task Allocation (task → machine):", np.round(gbest).astype(int))
print("Minimum Makespan:", gbest_fitness)
```

13. Build and train an LSTM to predict the next value in a univariate time series (e.g., daily sine wave + noise or stock-like synthetic series).

```
# LSTM for Next-Step Prediction (Univariate Time Series)
```

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.preprocessing import MinMaxScaler

# ----------------------------
# Generate sine wave + noise
# ----------------------------
np.random.seed(1)
t = np.arange(0, 200, 0.1)
series = np.sin(t) + 0.1 * np.random.randn(len(t))

# ----------------------------
# Prepare data
# ----------------------------
scaler = MinMaxScaler()
series_scaled = scaler.fit_transform(series.reshape(-1,1))

X, y = [], []
time_steps = 10
for i in range(len(series_scaled) - time_steps):
    X.append(series_scaled[i:i+time_steps])
    y.append(series_scaled[i+time_steps])

X = np.array(X)
y = np.array(y)

# ----------------------------
# Build LSTM model
# ----------------------------
model = Sequential([
    LSTM(50, activation='tanh', input_shape=(time_steps, 1)),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')

# ----------------------------
# Train model
```

```
# ----------------------------
history = model.fit(X, y, epochs=20, batch_size=32, verbose=1)

# ----------------------------
# Predict next value
# ----------------------------
last_sequence = series_scaled[-time_steps:].reshape(1, time_steps, 1)
next_value_scaled = model.predict(last_sequence)
next_value = scaler.inverse_transform(next_value_scaled)

print("Predicted next value:", next_value[0][0])

# ----------------------------
# Plot result
# ----------------------------
plt.plot(series, label='Original Series')
plt.scatter(len(series), next_value[0][0], color='red', label='Predicted Next')
plt.legend()
plt.show()
```

13b.

```
# Simple Univariate Time Series Prediction (Python – No TensorFlow)

import numpy as np

# Generate sine wave + noise
np.random.seed(1)
t = np.arange(0, 100, 0.1)
data = np.sin(t) + 0.1 * np.random.randn(len(t))

# Prepare sequences
step = 10
X, y = [], []
for i in range(len(data) - step):
   X.append(data[i:i+step])
   y.append(data[i+step])

X = np.array(X)
y = np.array(y)
```

```
# Simple weight-based sequence prediction (LSTM concept demo)
W = np.random.randn(step)
predictions = X @ W / step

print("Actual next value:", y[-1])
print("Predicted next value:", predictions[-1])
```

14. Use an LSTM to perform sentiment classification on a small text dataset (use IMDB subset or synthetic dataset).

```
# LSTM Sentiment Classification (Synthetic Dataset)
# PyTorch version – NO TensorFlow

import torch
import torch.nn as nn
import torch.optim as optim

# ----------------------
# Synthetic dataset
# ----------------------
sentences = [
   "I love this movie",
   "This film is great",
   "Amazing experience",
   "I hate this movie",
   "This film is terrible",
   "Worst experience"
]
labels = [1, 1, 1, 0, 0, 0]   # 1=positive, 0=negative

# ----------------------
# Tokenization
# ----------------------
vocab = {}
for s in sentences:
   for w in s.lower().split():
      if w not in vocab:
         vocab[w] = len(vocab) + 1

def encode(sentence):
   return [vocab.get(w, 0) for w in sentence.lower().split()]
```

```
X = [encode(s) for s in sentences]
y = torch.tensor(labels, dtype=torch.float)

# Padding
max_len = max(len(seq) for seq in X)
X_pad = [seq + [0]*(max_len - len(seq)) for seq in X]
X_tensor = torch.tensor(X_pad)

# ----------------------
# LSTM Model
# ----------------------
class LSTMClassifier(nn.Module):
  def __init__(self, vocab_size):
    super().__init__()
    self.embed = nn.Embedding(vocab_size + 1, 8, padding_idx=0)
    self.lstm = nn.LSTM(8, 16, batch_first=True)
    self.fc = nn.Linear(16, 1)
    self.sigmoid = nn.Sigmoid()

  def forward(self, x):
    x = self.embed(x)
    _, (h, _) = self.lstm(x)
    return self.sigmoid(self.fc(h[-1]))

model = LSTMClassifier(len(vocab))
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# ----------------------
# Training
# ----------------------
for epoch in range(200):
  optimizer.zero_grad()
  output = model(X_tensor).squeeze()
  loss = criterion(output, y)
  loss.backward()
  optimizer.step()

# ----------------------
```

```
# Prediction
# ----------------------
test_sentence = "I love this film"
test_encoded = encode(test_sentence)
test_encoded += [0] * (max_len - len(test_encoded))
test_tensor = torch.tensor([test_encoded])

prediction = model(test_tensor).item()
print("Sentiment score (0=negative, 1=positive):", prediction)
```

15. Implement SOM and use it for clustering and visualization of the Iris dataset (or high-dimensional synthetic data).

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Use only 2 features for easy visualization
X_vis = X[:, :2]

# Apply K-Means
kmeans = KMeans(n_clusters=3, random_state=0)
clusters = kmeans.fit_predict(X_vis)

# Plot clusters
plt.figure(figsize=(6,6))
plt.scatter(X_vis[:,0], X_vis[:,1], c=clusters)
plt.scatter(kmeans.cluster_centers_[:,0],
        kmeans.cluster_centers_[:,1],
        marker='X', s=200)

plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
```

```
plt.title("K-Means Clustering on Iris Dataset")
plt.grid()
plt.show()
```

15b.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Normalize data
X = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))

# SOM grid size
m, n = 7, 7
input_len = X.shape[1]

# Initialize weights
weights = np.random.random((m, n, input_len))

# Training parameters
epochs = 1000
initial_lr = 0.6
initial_radius = max(m, n) / 2

# Training SOM
for epoch in range(epochs):
    lr = initial_lr * np.exp(-epoch / epochs)
    radius = initial_radius * np.exp(-epoch / epochs)

    for sample in X:
        # Find Best Matching Unit (BMU)
        dist = np.linalg.norm(weights - sample, axis=2)
        bmu_x, bmu_y = np.unravel_index(np.argmin(dist), dist.shape)
```

```
        # Update weights of BMU and neighbors
        for i in range(m):
            for j in range(n):
                d = np.sqrt((i - bmu_x)**2 + (j - bmu_y)**2)
                if d <= radius:
                    influence = np.exp(-(d**2) / (2 * (radius**2)))
                    weights[i, j] += lr * influence * (sample - weights[i, j])

# Visualization
plt.figure(figsize=(7,7))
markers = ['o', 's', '^']
colors = ['red', 'green', 'blue']

for i, sample in enumerate(X):
    dist = np.linalg.norm(weights - sample, axis=2)
    bmu_x, bmu_y = np.unravel_index(np.argmin(dist), dist.shape)
    plt.scatter(bmu_x, bmu_y,
                c=colors[y[i]],
                marker=markers[y[i]],
                s=60)

plt.title("Self-Organizing Map (Proper Clustering)")
plt.grid()
plt.show()
```

16. Implement a Hopfield network to store and recall binary patterns; demonstrate recall from noisy inputs.

```
import numpy as np
import matplotlib.pyplot as plt

# Convert 0/1 to -1/1
def to_bipolar(pattern):
    return np.where(pattern == 0, -1, 1)

# Hopfield Network class
class HopfieldNetwork:
    def __init__(self, num_neurons):
        self.num_neurons = num_neurons
        self.weights = np.zeros((num_neurons, num_neurons))
```

```
    def train(self, patterns):
        for p in patterns:
            self.weights += np.outer(p, p)
        np.fill_diagonal(self.weights, 0)
        self.weights /= len(patterns)

    def recall(self, pattern, steps=10):
        s = pattern.copy()
        for _ in range(steps):
            for i in range(self.num_neurons):
                net_input = np.dot(self.weights[i], s)
                s[i] = 1 if net_input >= 0 else -1
        return s

# Define binary patterns (5x5)
patterns = [
    np.array([
        1,0,0,0,1,
        0,1,0,1,0,
        0,0,1,0,0,
        0,1,0,1,0,
        1,0,0,0,1
    ]),
    np.array([
        0,1,1,1,0,
        1,0,0,0,1,
        1,0,0,0,1,
        1,0,0,0,1,
        0,1,1,1,0
    ])
]

# Convert patterns to bipolar
patterns = [to_bipolar(p) for p in patterns]

# Create and train network
hopfield = HopfieldNetwork(num_neurons=25)
hopfield.train(patterns)

# Create noisy version of first pattern
```

```
noisy_pattern = patterns[0].copy()
noise_indices = np.random.choice(25, 6, replace=False)
noisy_pattern[noise_indices] *= -1

# Recall pattern
recalled_pattern = hopfield.recall(noisy_pattern)

# Plot patterns
def plot_pattern(pattern, title, pos):
  plt.subplot(1, 3, pos)
  plt.imshow(pattern.reshape(5,5), cmap='gray')
  plt.title(title)
  plt.axis('off')

plt.figure(figsize=(9,3))
plot_pattern(patterns[0], "Original Pattern", 1)
plot_pattern(noisy_pattern, "Noisy Input", 2)
plot_pattern(recalled_pattern, "Recalled Pattern", 3)
plt.show()
```

17. Implement BAM to associate input patterns A to output patterns B (e.g., letter-to-digit mapping), and recall outputs given noisy inputs.

```
import numpy as np

# ----------------------------
# Step 1: Define input patterns (Letters)
# Using bipolar values: 1 and -1
# ----------------------------

A = np.array([
  [ 1, -1,  1, -1],   # Pattern A1 (Letter A)
  [-1,  1, -1,  1],   # Pattern A2 (Letter B)
  [ 1,  1, -1, -1]    # Pattern A3 (Letter C)
])

# ----------------------------
# Step 2: Define output patterns (Digits)
# ----------------------------
```

```
B = np.array([
   [ 1, -1, -1],   # Digit 1
   [-1,  1, -1],   # Digit 2
   [-1, -1,  1]    # Digit 3
])

# ----------------------------
# Step 3: Train BAM (Weight matrix)
# W = sum of outer products
# ----------------------------

W = np.zeros((A.shape[1], B.shape[1]))

for i in range(len(A)):
   W += np.outer(A[i], B[i])

print("Weight Matrix W:\n", W)

# ----------------------------
# Step 4: Recall with noisy input
# ----------------------------

# Noisy version of A1
A_noisy = np.array([1, -1, -1, -1])

# Forward recall (A → B)
B_recalled = np.sign(A_noisy @ W)

# Handle zero values
B_recalled[B_recalled == 0] = 1

print("\nNoisy Input A:", A_noisy)
print("Recalled Output B:", B_recalled)

# ----------------------------
# Step 5: Backward recall (B → A)
# ----------------------------

A_recalled = np.sign(B_recalled @ W.T)
A_recalled[A_recalled == 0] = 1
```

```
print("Recalled Input A:", A_recalled)
```